نقشه‌ی راه کامل پروژه‌ی React

از صفر تا صد؛ مرحله‌به‌مرحله و قابل استفاده برای همه‌ی پروژه‌ها

Vite · React Router · Hooks · API · Deploy

فهرست و روش استفاده

این سند یک **نقشه‌ی راه ثابت** است. هر پروژه‌ی React، چه یک اپ فیلم، چه فروشگاه یا داشبورد، همین مراحل را به همین ترتیب طی می‌کند. فقط محتوای هر مرحله فرق می‌کند. پروژه‌ی اول را مرحله‌به‌مرحله جلو ببر و در پروژه‌های بعدی از چک‌لیست انتهای سند استفاده کن.

فهرست مراحل

- مرحله ۰: ایده و برنامه‌ریزی (قبل از نوشتن کد)
- مرحله ۱: آماده‌سازی محیط
- مرحله ۲: ساخت پروژه با Vite
- مرحله ۳: ساختار پوشه‌ها
- مرحله ۴: ساخت UI ایستا (کامپوننت‌ها، JSX، props)
- مرحله ۵: استایل‌دهی
- مرحله ۶: مسیریابی (React Router)
- مرحله ۷: State و مدیریت داده‌ی داخلی
- مرحله ۸: Effect و دریافت داده از API
- مرحله ۹: فرم‌ها و رویدادها
- مرحله ۱۰: State سراسری (Context) و هوک سفارشی
- مرحله ۱۱: بهینه‌سازی و هوک‌های دیگر
- مرحله ۱۲: کیفیت کد و دیباگ
- مرحله ۱۳: دسترس‌پذیری، ریسپانسیو و پرداخت نهایی
- مرحله ۱۴: تست
- مرحله ۱۵: Git و GitHub
- مرحله ۱۶: Build و Deploy
- اصول طلایی React
- اشتباهات رایج و راه‌حلشان
- چک‌لیست هر پروژه
- دستورات و میانبرها
- بعد از پروژه‌ی اول چه یاد بگیرم؟

> یک قانون برای کل مسیر: **هر بار فقط یک مرحله‌ی کوچک بساز و همان لحظه در مرورگر ببین کار می‌کند یا نه.** اگر چند ساعت کد بنویسی و بعد اجرا کنی، پیدا کردن خطا بسیار سخت می‌شود.

مرحله ۰: ایده و برنامه‌ریزی

قبل از باز کردن VS Code، روی کاغذ یا در یک فایل متنی به این سؤال‌ها جواب بده. ده دقیقه برنامه‌ریزی چند ساعت دوباره‌کاری را حذف می‌کند.

۱. مسئله را در یک جمله بنویس

مثال: «کاربر بتواند فیلم‌های محبوب را ببیند، جستجو کند و فیلم‌های موردعلاقه‌اش را ذخیره کند.»

۲. قابلیت‌ها را به دو دسته تقسیم کن

- **MVP (حداقل نسخه‌ی قابل استفاده):** فقط چیزهایی که بدون آن‌ها برنامه معنی ندارد.
- **بعداً:** همه‌ی ایده‌های اضافه (دارک‌مود، لاگین، امتیازدهی و ...).

فقط MVP را بساز. بقیه را بعد از تمام شدن MVP اضافه کن.

۳. صفحه‌ها (Pages) و آدرس‌ها را مشخص کن

صفحه	آدرس	کار
Home	`/`	نمایش فیلم‌های محبوب و جستجو
Favorites	`/favorites`	لیست فیلم‌های موردعلاقه
NotFound	`*`	صفحه‌ی ۴۰۴

۴. کامپوننت‌های تکراری را پیدا کن

هر چیزی که بیش از یک بار تکرار می‌شود یا مستقل است، یک کامپوننت می‌شود: `NavBar`، `MovieCard`، `SearchForm`، `Loader`.

۵. داده از کجا می‌آید؟

- داده‌ی ثابت داخل کد (برای شروع و تست ظاهر)
- API بیرونی (مثل TMDB)
- ذخیره در `localStorage` مرورگر
- بک‌اند خودت (بعداً)

۶. طرح کاغذی (Wireframe)

با دست یا با Figma، چیدمان هر صفحه را بکش. لازم نیست زیبا باشد، فقط بدان هر چیز کجا قرار می‌گیرد.

> چرا مهم است؟ وقتی بدانی صفحه از چه کامپوننت‌هایی ساخته می‌شود و داده از کجا می‌آید، دیگر وسط کار نمی‌مانی که «حالا چه کنم؟».

مرحله ۱: آماده‌سازی محیط

فقط یک بار برای همیشه نصب می‌شود.

ابزار	کاربرد	بررسی نصب
Node.js (نسخه‌ی LTS)	اجرای ابزارها و npm	`node -v`
npm	نصب پکیج‌ها (همراه Node)	`npm -v`
Git	کنترل نسخه	`git --version`
VS Code	ویرایشگر کد	-
Edge یا Chrome	مرورگر و DevTools	-

افزونه‌های مفید VS Code

- **ES7+ React/Redux Snippets:** با نوشتن `rafce` کل ساختار کامپوننت ساخته می‌شود.
- **Prettier:** فرمت خودکار کد.
- **Auto Rename Tag:** با تغییر تگ باز، تگ بسته هم تغییر می‌کند.
- **ESLint یا Oxc:** نمایش خطاها هنگام نوشتن.

افزونه‌ی مرورگر

React Developer Tools را در Chrome یا Edge نصب کن. با آن می‌توانی درخت کامپوننت‌ها، props و state را ببینی.

مرحله ۲: ساخت پروژه با Vite

دستورها

```
npm create vite@latest my-app -- --template react
cd my-app
npm install
npm run dev
```

بعد از `npm run dev` یک آدرس مثل `http://localhost:5173` نشان داده می‌شود. آن را در مرورگر باز کن. **همان آدرسی را باز کن که در ترمینال چاپ شده**، چون اگر پورت اشغال باشد Vite پورت دیگری برمی‌دارد.

فایل‌های مهم پروژه

فایل / پوشه	کار
`index.html`	تنها صفحه‌ی HTML؛ یک `div` با id برابر `root` دارد و فایل `main.jsx` را لود می‌کند
`src/main.jsx`	نقطه‌ی شروع؛ کامپوننت `App` را داخل `root` رندر می‌کند
`src/App.jsx`	کامپوننت اصلی؛ معمولاً مسیرها (Routes) اینجا هستند
`package.json`	اسم پروژه، پکیج‌ها و اسکریپت‌ها (`dev` ، `build` ، `lint`)
`vite.config.js`	تنظیمات Vite
`public/`	فایل‌های ثابت که بدون تغییر سرو می‌شوند (favicon و ...)
`node_modules/`	پکیج‌های نصب‌شده؛ **هرگز دستکاری یا commit نمی‌شود**
`.gitignore`	فایل‌هایی که Git باید نادیده بگیرد

تمیز کردن قالب اولیه

1. محتوای نمونه‌ی `App.jsx` را پاک کن و فقط یک `<div>` ساده بگذار.
2. فایل‌های بلااستفاده‌ی `assets` و `App.css` را حذف یا خالی کن.
3. در `index.html` عنوان `<title>` را به اسم پروژه‌ات تغییر بده.

اولین commit

```
git init
git add .
git commit -m "Initial commit: Vite + React"
```

مرحله ۳: ساختار پوشه‌ها

از همان اول یک ساختار منظم بساز تا پروژه با بزرگ شدن به هم نریزد.

```
src/
├── components/   کامپوننت‌های قابل استفاده‌ی مجدد (NavBar, MovieCard)
├── pages/        هر صفحه یک فایل (Home, Favorites)
├── services/     کد ارتباط با API (api.js)
├── contexts/     state سراسری (MovieContext.jsx)
├── hooks/        هوک‌های سفارشی (useFetch.js)
├── css/          فایل‌های استایل
├── utils/        تابع‌های کمکی (فرمت تاریخ و ...)
├── assets/       عکس و آیکن
├── App.jsx
└── main.jsx
```

قوانین نام‌گذاری

- فایل و اسم کامپوننت: **PascalCase** (مثل `MovieCard.jsx`).
- تابع و متغیر: **camelCase** (مثل `handleSearch`).
- ثابت‌ها: `UPPER_CASE` (مثل `BASE_URL`).
- هر فایل فقط یک کامپوننت اصلی دارد و با `export default` خارج می‌شود.

> **حساسیت به حروف بزرگ و کوچک:** ویندوز بین `Navbar.jsx` و `NavBar.jsx` فرق نمی‌گذارد، ولی سرور Linux (مثل Vercel و Netlify) فرق می‌گذارد. اگر اسم فایل `Navbar.jsx` باشد و import شما `NavBar` باشد، روی کامپیوتر خودت کار می‌کند ولی بعد از Deploy خطای build می‌دهد. اسم فایل و import را دقیقاً یکی کن.

Pages در برابر Components

- **Page:** یک صفحه‌ی کامل که به یک آدرس (Route) وصل است. داده‌ی خودش را می‌گیرد و چند کامپوننت را کنار هم می‌چیند.
- **Component:** یک تکه‌ی کوچک قابل استفاده‌ی مجدد که معمولاً داده را از والد با props می‌گیرد.

مثال: `MovieCard` یک کامپوننت است و بدون prop به اسم `movie` کار نمی‌کند، پس نباید مستقیماً به یک Route وصل شود.

مرحله ۴: ساخت UI ایستا

اول بدون API، بدون state و با داده‌ی ثابت، ظاهر را بساز.

قوانین JSX

1. هر کامپوننت باید **فقط یک عنصر والد** برگرداند (یا از `<>...</>` استفاده کن).
2. به جای `class` از `className` استفاده کن.
3. همه‌ی تگ‌ها باید بسته شوند: `<img />` و `<input />` .
4. ویژگی‌ها camelCase هستند: `onClick` ، `onChange` ، `htmlFor` .
5. کد جاوااسکریپت داخل `{ }` می‌رود.
6. اسم کامپوننت حتماً با **حرف بزرگ** شروع شود، وگرنه React آن را تگ HTML می‌داند.
7. برای چند خط JSX، آن را داخل **پرانتز** بعد از `return` بگذار.

```
function NavBar() {
  return (
    <nav className="navbar">
      <div className="navbar-brand">Movie App</div>
      <div className="navbar-links">
        <a href="/">Home</a>
      </div>
    </nav>
  );
}

export default NavBar;
```

Props: دادن اطلاعات به کامپوننت

```
function MovieCard({ movie }) {
  return (
    <div className="movie-card">
      <h3>{movie.title}</h3>
      <p>{movie.release_date}</p>
    </div>
  );
}

// استفاده
<MovieCard movie={{ title: "The Matrix", release_date: "1999" }} />
```

props فقط **خواندنی** هستند. هرگز مقدار prop را داخل کامپوننت تغییر نده.

نمایش لیست با **map** و **key**

```
const movies = [
  { id: 1, title: "John Wick" },
  { id: 2, title: "The Matrix" },
];

<div className="movies-grid">
  {movies.map((movie) => (
    <MovieCard movie={movie} key={movie.id} />
  ))}
</div>
```

`key` باید **یکتا و ثابت** باشد (مثل `id`). از اندیس آرایه (`index`) استفاده نکن، مگر اینکه لیست هرگز عوض نشود.

نمایش شرطی

```
{loading && <p>Loading...</p>}
{error ? <p>{error}</p> : <MovieList />}
{movies.length === 0 && <p>No movies found</p>}
```

مرحله ۵: استایل‌دهی

روش پیشنهادی برای شروع

- یک فایل `index.css` برای استایل‌های **سراسری** (فونت، رنگ پس‌زمینه، reset).
- برای هر کامپوننت یا صفحه یک فایل CSS جدا و import آن در همان کامپوننت.

```
import "../css/MovieCard.css";
```

reset ساده در index.css

```
* { margin: 0; padding: 0; box-sizing: border-box; }
body { font-family: system-ui, sans-serif; }
```

ابزارهای مهم چیدمان

- **Flexbox:** چیدمان یک‌بعدی (ردیف یا ستون)، مناسب نوار ناوبری و فرم.
- **Grid:** چیدمان دوبعدی، مناسب شبکه‌ی کارت‌ها.

```
.movies-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(180px, 1fr));
  gap: 1.5rem;
}
```

ریسپانسیو

اول برای موبایل بنویس، بعد با `@media` برای صفحه‌های بزرگ‌تر اصلاح کن.

```
@media (min-width: 768px) {
  .navbar { padding: 1rem 2rem; }
}
```

اشتباه رایج: نام `className` در JSX با نام کلاس در CSS باید **دقیقاً یکی** باشد. اگر در JSX نوشته باشی `movie-grid` ولی در CSS `.movies-grid` باشد، استایل اعمال نمی‌شود. وقتی استایلی کار نمی‌کند، با راست‌کلیک و Inspect در مرورگر ببین کلاس در عنصر هست و CSS به آن رسیده یا نه.

مرحله ۶: مسیریابی (React Router)

نصب

```
npm install react-router-dom
```

مرحله ۱: BrowserRouter را در main.jsx بگذار

```
import { BrowserRouter } from "react-router-dom";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>
);
```

اگر BrowserRouter نباشد، خطای `useRoutes() may be used only in the context of a <Router>` می‌گیری و صفحه سفید می‌شود.

مرحله ۲: Routeها را در App.jsx تعریف کن

```
import { Routes, Route } from "react-router-dom";

function App() {
  return (
    <div>
      <NavBar />
      <main className="main-content">
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/favorites" element={<Favorites />} />
          <Route path="*" element={<NotFound />} />
        </Routes>
      </main>
    </div>
  );
}
```

لینک‌ها

به جای `<a href>` از `Link` یا `NavLink` استفاده کن تا صفحه reload نشود.

```
import { Link, NavLink } from "react-router-dom";

<Link to="/favorites">Favorites</Link>
<NavLink to="/" className={({ isActive }) => (isActive ? "active" : "")}>Home</NavLink>
```

مسیر پویا (مثلاً صفحه‌ی جزئیات)

```
<Route path="/movie/:id" element={<MovieDetails />} />

// داخل MovieDetails
import { useParams } from "react-router-dom";
const { id } = useParams();
```

پرش برنامه‌نویسی‌شده

```
import { useNavigate } from "react-router-dom";
const navigate = useNavigate();
navigate("/favorites");
```

مرحله ۷: State و مدیریت داده‌ی داخلی

state داده‌ای است که با تغییرش، کامپوننت دوباره رندر می‌شود و صفحه به‌روز می‌شود.

useState

```
import { useState } from "react";

const [searchQuery, setSearchQuery] = useState("");
//     مقدار اولیه         تابع تغییر     مقدار فعلی
```

قوانین state

1. **هرگز state را مستقیم تغییر نده.** همیشه از تابع set استفاده کن.
2. برای آرایه و آبجکت، **نسخه‌ی جدید** بساز (`...` یا `map` و `filter`)، نه اینکه نسخه‌ی قبلی را تغییر دهی.
3. اگر مقدار جدید به مقدار قبلی بستگی دارد، از **تابع** استفاده کن: `setCount(prev => prev + 1)` .
4. چیزی که می‌توان از state دیگر **محاسبه** کرد، state جدا نمی‌خواهد.

```
// اضافه کردن
setItems((prev) => [...prev, newItem]);

// حذف
setItems((prev) => prev.filter((item) => item.id !== id));

// ویرایش
setItems((prev) => prev.map((item) => (item.id === id ? { ...item, done: true } : item)));

// آبجکت
setUser((prev) => ({ ...prev, name: "Ali" }));
```

State را کجا بگذارم؟

قانون: state را در **نزدیک‌ترین والد مشترک** کامپوننت‌هایی بگذار که به آن نیاز دارند (به این کار Lifting State Up می‌گویند).
اگر چند صفحه‌ی مختلف به آن نیاز دارند، از Context استفاده کن (مرحله ۱۰).

جریان داده یک‌طرفه است

داده از والد به فرزند با **props** می‌رود. فرزند برای تغییر دادن، یک **تابع** از والد می‌گیرد و صدایش می‌زند.

```
function Parent() {
  const [count, setCount] = useState(0);
  return <Child count={count} onIncrement={() => setCount((c) => c + 1)} />;
}
```

مرحله ۸: Effect و دریافت داده از API

useEffect

برای کارهایی است که **بیرون از رندر** انجام می‌شوند: دریافت داده، تایمر، اتصال به مرورگر.

```
useEffect(() => {
  // کار
  return () => { /* (اختیاری) پاکسازی */ };
}, [dependencies]);
```

آرایه‌ی وابستگی	چه زمانی اجرا می‌شود
`[]`	فقط یک بار بعد از اولین رندر
`[query]`	بعد از اولین رندر و هر بار که `query` تغییر کند
بدون آرایه	بعد از هر رندر (تقریباً همیشه اشتباه است)

> در حالت توسعه، `StrictMode` عمداً effect را دو بار اجرا می‌کند. این فقط در dev است و در build نهایی نیست، پس نگران نباش.

الگوی استاندارد دریافت داده (حفظ کن)

سه state بساز: داده، در حال بارگذاری، خطا.

```
const [movies, setMovies] = useState([]);
const [loading, setLoading] = useState(true);
const [error, setError] = useState(null);

useEffect(() => {
  const loadMovies = async () => {
    try {
      const data = await getPopularMovies();
      setMovies(data);
    } catch (err) {
      console.error(err);
      setError("Failed to load movies...");
    } finally {
      setLoading(false);
    }
  };
  loadMovies();
}, []);
```

در JSX:

```
{error && <div className="error-message">{error}</div>}
{loading ? (
  <div className="loading">Loading...</div>
) : (
  <div className="movies-grid">
    {movies.map((m) => <MovieCard movie={m} key={m.id} />)}
  </div>
)}
```

> **هر `try` حتماً `catch` یا `finally` می‌خواهد.** و تابع async که داخل effect تعریف می‌کنی باید **صدا زده شود** (`loadMovies()`)، وگرنه هرگز اجرا نمی‌شود.

فایل سرویس API

کد fetch را داخل کامپوننت ننویس. در `src/services/api.js` بگذار:

```
const API_KEY = import.meta.env.VITE_TMDB_KEY;
const BASE_URL = "https://api.themoviedb.org/3";

export const getPopularMovies = async () => {
  const response = await fetch(`${BASE_URL}/movie/popular?api_key=${API_KEY}`);
  if (!response.ok) throw new Error("Request failed: " + response.status);
  const data = await response.json();
  return data.results;
};

export const searchMovies = async (query) => {
  const response = await fetch(
    `${BASE_URL}/search/movie?api_key=${API_KEY}&query=${encodeURIComponent(query)}`
  );
  if (!response.ok) throw new Error("Request failed: " + response.status);
  const data = await response.json();
  return data.results;
};
```

نکته‌ی مهم: `fetch` برای خطاهای ۴۰۴ یا ۵۰۰ خودش خطا **پرتاب نمی‌کند**. حتماً `response.ok` را بررسی کن.

متغیرهای محیطی و کلید API

1. فایل `.env` را در ریشه‌ی پروژه بساز (کنار `package.json`) و کلید را داخلش بنویس (مثال: `VITE_TMDB_KEY=your_key_here`).
2. در کد با `import.meta.env.VITE_TMDB_KEY` بخوان. نام باید با `VITE_` شروع شود.
3. `.env` را در `.gitignore` بگذار تا روی GitHub نرود.
4. بعد از تغییر `.env` ، سرور dev را دوباره اجرا کن.

هر چیزی که در فرانت‌اند باشد، در نهایت برای کاربر قابل دیدن است. `.env` فقط کلید را از GitHub پنهان می‌کند، نه از مرورگر. برای کلیدهای واقعاً محرمانه باید از بک‌اند استفاده کنی.

جستجو با API

```
const handleSearch = async (e) => {
  e.preventDefault();
  if (!searchQuery.trim()) return;
  if (loading) return;
  setLoading(true);
  try {
    const results = await searchMovies(searchQuery);
    setMovies(results);
    setError(null);
  } catch (err) {
    console.error(err);
    setError("Failed to search movies...");
  } finally {
    setLoading(false);
  }
};
```

مرحله ۹: فرم‌ها و رویدادها

input کنترل‌شده (Controlled Input)

مقدار input همیشه از state می‌آید و با هر تغییر، state به‌روز می‌شود.

```
const [searchQuery, setSearchQuery] = useState("");

<form onSubmit={handleSearch} className="search-form">
  <input
    type="text"
    placeholder="Search for movies..."
    value={searchQuery}
    onChange={(e) => setSearchQuery(e.target.value)}
  />
  <button type="submit">Search</button>
</form>
```

قوانین

1. در `onSubmit` همیشه `e.preventDefault()` بزن، وگرنه صفحه reload می‌شود.
2. دکمه‌ی ارسال را `type="submit"` بگذار و `onSubmit` را روی `form` بگذار، نه `onClick` روی دکمه (تا Enter هم کار کند).
3. در `onClick` تابع را **صدا نزن**، **پاس بده**: `onClick={handle}` درست است ولی `onClick={handle()}` اشتباه است. اگر آرگومان داری: `onClick={() => handle(id)}` .

فرم با چند فیلد

```
const [form, setForm] = useState({ name: "", email: "" });

const handleChange = (e) => {
  setForm((prev) => ({ ...prev, [e.target.name]: e.target.value }));
};

<input name="name" value={form.name} onChange={handleChange} />
<input name="email" value={form.email} onChange={handleChange} />
```

اعتبارسنجی ساده

قبل از ارسال، مقدارها را بررسی کن و پیام خطا را در state نگه دار و زیر فیلد نشان بده.

مرحله ۱۰: State سراسری (Context) و هوک سفارشی

وقتی چند صفحه‌ی دور از هم به یک داده نیاز دارند (مثل «فیلم‌های موردعلاقه» که هم در Home و هم در Favorites لازم است)، به جای فرستادن props از چند لایه، از **Context** استفاده کن.

ساخت Context

`src/contexts/MovieContext.jsx` :

```
import { createContext, useContext, useState, useEffect } from "react";

const MovieContext = createContext();

export const useMovieContext = () => useContext(MovieContext);

export function MovieProvider({ children }) {
  const [favorites, setFavorites] = useState(() => {
    const stored = localStorage.getItem("favorites");
    return stored ? JSON.parse(stored) : [];
  });

  useEffect(() => {
    localStorage.setItem("favorites", JSON.stringify(favorites));
  }, [favorites]);

  const addToFavorites = (movie) => setFavorites((prev) => [...prev, movie]);
  const removeFromFavorites = (id) =>
    setFavorites((prev) => prev.filter((m) => m.id !== id));
  const isFavorite = (id) => favorites.some((m) => m.id === id);

  return (
    <MovieContext.Provider
      value={{ favorites, addToFavorites, removeFromFavorites, isFavorite }}
    >
      {children}
    </MovieContext.Provider>
  );
}
```

وصل کردن Provider

```
<BrowserRouter>
  <MovieProvider>
    <App />
  </MovieProvider>
</BrowserRouter>
```

استفاده در هر کامپوننت

```
const { isFavorite, addToFavorites, removeFromFavorites } = useMovieContext();
const favorite = isFavorite(movie.id);

function onFavoriteClick(e) {
  e.preventDefault();
  if (favorite) removeFromFavorites(movie.id);
  else addToFavorites(movie);
}
```

localStorage

داده را در مرورگر کاربر نگه می‌دارد تا با رفرش پاک نشود. فقط **رشته** ذخیره می‌کند، پس با `JSON.stringify` ذخیره و با `JSON.parse` بخوان. مقدار اولیه‌ی `useState` را به شکل **تابع** بده (مثل کد بالا) تا فقط یک بار خوانده شود.

هوک سفارشی (Custom Hook)

اگر یک الگوی منطقی را در چند جا تکرار می‌کنی، آن را در یک تابع که اسمش با `use` شروع می‌شود جدا کن:

```
// src/hooks/useFetch.js
import { useState, useEffect } from "react";

export function useFetch(fetcher) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    let ignore = false;
    fetcher()
      .then((d) => { if (!ignore) setData(d); })
      .catch((e) => { if (!ignore) setError(e.message); })
      .finally(() => { if (!ignore) setLoading(false); });
    return () => { ignore = true; };
  }, []);

  return { data, loading, error };
}
```

> چه موقع Context؟ فقط وقتی داده واقعاً در چند جای دور از هم لازم است. برای داده‌ای که فقط یک صفحه استفاده می‌کند، state محلی کافی است. اگر برنامه خیلی بزرگ شد، بعداً Zustand یا Redux Toolkit را یاد بگیر.

مرحله ۱۱: بهینه‌سازی و هوک‌های دیگر

اول درست، بعد سریع

بهینه‌سازی زودهنگام ممنوع است. اول برنامه را درست کار بینداز. فقط اگر واقعاً کندی دیدی، بهینه‌سازی کن.

هوک / ابزار	کاربرد	مثال
`useRef`	نگه داشتن مقدار بدون رندر مجدد، یا دسترسی به عنصر DOM	فوکوس روی input
`useMemo`	ذخیره‌ی نتیجه‌ی یک محاسبه‌ی سنگین	فیلتر لیست بزرگ
`useCallback`	ثابت نگه داشتن یک تابع بین رندرها	تابع پاس‌داده‌شده به کامپوننت memo شده
`React.memo`	جلوگیری از رندر مجدد کامپوننت وقتی props عوض نشده	کارت‌های لیست بلند
`lazy` + `Suspense`	لود کردن صفحه‌ها فقط وقتی لازم‌اند	کاهش حجم بسته‌ی اولیه

مثال useRef

```
const inputRef = useRef(null);
<input ref={inputRef} />
<button onClick={() => inputRef.current.focus()}>Focus</button>
```

مثال lazy

```
import { lazy, Suspense } from "react";
const Favorites = lazy(() => import("./pages/Favorites"));

<Suspense fallback={<p>Loading...</p>}>
  <Routes>...</Routes>
</Suspense>
```

تأخیر در جستجو (Debounce)

برای جستجوی لحظه‌ای، با هر حرف درخواست نفرست. با `setTimeout` در effect کمی صبر کن:

```
useEffect(() => {
  const timer = setTimeout(() => runSearch(query), 500);
  return () => clearTimeout(timer);
}, [query]);
```

مرحله ۱۲: کیفیت کد و دیباگ

ابزارها

- **Lint:** `npm run lint` اشتباهات و متغیرهای بلااستفاده را نشان می‌دهد.
- **Build:** `npm run build` اگر خطای نحوی (Syntax) داشته باشی همین‌جا فاصله می‌گیرد؛ قبل از هر Deploy اجرایش کن.
- **Prettier:** فرمت یکدست کد.

خواندن پیام خطا

1. پیام را **کامل** بخوان؛ نوع خطا و نام فایل و شماره‌ی خط در آن است.
2. به فایل و خط گفته‌شده برو. معمولاً مشکل همان‌جا یا چند خط بالاتر است.
3. خطای اول را درست کن؛ بقیه‌ی خطاها اغلب نتیجه‌ی همان خطای اول‌اند.

ابزارهای دیباگ

ابزار	کاربرد
`console.log`	دیدن مقدار متغیر در لحظه‌ی اجرا
تب Console در DevTools	خطاهای زمان اجرا
تب Network	بررسی درخواست‌های API (وضعیت ۲۰۰، ۴۰۴ و پاسخ)
تب Elements	بررسی HTML و CSS اعمال‌شده
React DevTools	بررسی props و state کامپوننت‌ها

قوانین هوک‌ها (باید رعایت شوند)

1. هوک‌ها را فقط در **بالاترین سطح** کامپوننت صدا بزن؛ نه داخل `if`، حلقه یا تابع تو در تو.
2. هوک‌ها را فقط داخل **کامپوننت** یا **هوک سفارشی** صدا بزن.
3. اسم هوک سفارشی باید با `use` شروع شود.

مدیریت خطا در رابط کاربری

همیشه سه حالت را نشان بده: **در حال بارگذاری**، **خطا**، **داده‌ی خالی**. کاربر نباید با صفحه‌ی خالی و بی‌توضیح مواجه شود.

مرحله ۱۳: دسترس‌پذیری، ریسپانسیو و پرداخت نهایی

دسترس‌پذیری (Accessibility)

- از تگ‌های معنایی استفاده کن: `header` ، `nav` ، `main` ، `footer` ، `section` . در هر صفحه فقط یک `main` باشد.
- برای هر عکس `alt` بنویس.
- برای هر input یک `label` بگذار (با `htmlFor` و `id`).
- چیزی که کلیک می‌شود باید `button` یا `a` باشد، نه `div` .
- کنتراست رنگ متن و پس‌زمینه را کافی نگه دار.

ریسپانسیو

برنامه را در مرورگر با ابزار شبیه‌ساز موبایل (Ctrl+Shift+M در DevTools) در عرض‌های ۳۷۵، ۷۶۸ و ۱۲۰۰ پیکسل ببین.

تجربه‌ی کاربری

- حالت Loading و Error و خالی را طراحی کن.
- روی دکمه‌ها و کارت‌ها حالت hover و focus بگذار.
- عنوان تب (`title`) و favicon را تنظیم کن.
- پیام‌ها واضح و کوتاه باشند.

مرحله ۱۴: تست

تست دستی (همیشه)

قبل از هر Deploy، همه‌ی سناریوها را خودت امتحان کن:

- هر صفحه باز می‌شود؟
- هر دکمه و لینک کار می‌کند؟
- حالت خطا (مثلاً اینترنت قطع) رفتار درست دارد؟
- ریسپانسیو در موبایل درست است؟
- بعد از رفرش داده‌ها (مثل علاقه‌مندی‌ها) باقی می‌مانند؟

تست خودکار (برای شروع جدی)

```
npm install -D vitest jsdom @testing-library/react @testing-library/jest-dom
```

در `vite.config.js` محیط تست را jsdom کن:

```
export default defineConfig({
  plugins: [react()],
  test: { environment: "jsdom", globals: true },
});
```

نمونه‌ی تست:

```
import { render, screen } from "@testing-library/react";
import MovieCard from "./MovieCard";

test("shows movie title", () => {
  render(<MovieCard movie={{ id: 1, title: "Matrix", release_date: "1999-03-31" }} />);
  expect(screen.getByText("Matrix")).toBeTruthy();
});
```

اجرا: `npx vitest` .

مرحله ۱۵: Git و GitHub

قوانین کار با Git

- **کم‌کم و زیاد commit کن**؛ هر بار که یک قابلیت کوچک تمام شد.
- پیام commit واضح باشد: `Add favorites page`، `Fix search bug`.
- هرگز `node_modules` و `.env` را commit نکن (در `.gitignore` باشند).
- برای هر قابلیت‌ی بزرگ یک شاخه (branch) جدا بساز.

دستورهای روزمره

```
git status                    # ببین چه چیزی تغییر کرده
git add .                     # همه‌ی تغییرها را آماده‌ی commit کن
git commit -m "Add favorites" # ثبت تغییرها
git switch -c feature/search  # ساخت و رفتن به شاخه‌ی جدید
git switch main               # برگشت به شاخه‌ی اصلی
git merge feature/search      # ادغام شاخه
git log --oneline             # تاریخچه‌ی کوتاه
```

گذاشتن پروژه روی GitHub

1. در GitHub یک repository خالی بساز.
2. دستورهایی که GitHub نشان می‌دهد را اجرا کن:

```
git remote add origin https://github.com/USERNAME/REPO.git
git branch -M main
git push -u origin main
```

از آن به بعد با `git push` تغییرها را بفرست.

مرحله ۱۶: Build و Deploy

ساخت نسخه‌ی نهایی

```
npm run build      # ساخت پوشه‌ی dist
npm run preview    # اجرای نسخه‌ی build شده برای تست محلی
```

پوشه‌ی `dist` همان چیزی است که روی سرور می‌رود. قبل از Deploy حتماً `npm run build` بدون خطا تمام شود.

سرویس‌های رایگان

سرویس	نکته
Vercel	ساده‌ترین؛ repository را از GitHub وصل کن
Netlify	مشابه Vercel
GitHub Pages	نیاز به تنظیم `base` در `vite.config.js`

تنظیمات لازم

1. **متغیرهای محیطی:** `VITE_TMDB_KEY` را در پنل سرویس (Environment Variables) اضافه کن؛ چون `.env` به GitHub نمی‌رود.
2. **بازنویسی مسیرها (SPA):** چون React Router در مرورگر مسیر می‌سازد، رفرش در آدرسی مثل `/favorites` روی سرور ۴۰۴ می‌دهد، مگر اینکه همه‌ی مسیرها به `index.html` برگردند.

برای Vercel فایل `vercel.json` در ریشه‌ی پروژه:

```
{ "rewrites": [{ "source": "/(.*)", "destination": "/" }] }
```

برای Netlify فایل `public/_redirects` با این محتوا:

```
/*  /index.html  200
```

ادامه‌ی تنظیمات:

3. **حروف بزرگ و کوچک:** اسم فایل‌ها و import ها دقیقاً یکی باشند (مرحله ۳).
4. **GitHub Pages:** در `vite.config.js` مقدار `base: "/REPO-NAME/"` را بگذار.

بعد از Deploy

آدرس را در موبایل و مرورگر دیگر باز کن، همه‌ی صفحه‌ها را امتحان کن و Console را از نظر خطا ببین.

اصول طلایی React

1. **UI تابعی از state است.** state را عوض کن، React خودش صفحه را به‌روز می‌کند. DOM را دستی تغییر نده.
2. **جریان داده یک‌طرفه است:** props از بالا به پایین، رویدادها از پایین به بالا.
3. **state هرگز مستقیم تغییر نمی‌کند.** همیشه با تابع set و با نسخه‌ی جدید.
4. **منبع حقیقت یگانه:** هر داده فقط در یک جا (state) زندگی کند، نه کپی در چند جا.
5. **هر کامپوننت یک وظیفه** دارد. اگر بزرگ شد، تکه‌اش کن.
6. **کامپوننت‌ها خالص باشند:** با یک props و state، همیشه یک خروجی بدهند و کار جانبی (fetch و ...) در رندر نکنند. کار جانبی جایش `useEffect` یا event handler است.
7. **key یکتا و ثابت** برای هر عنصر لیست.
8. **قوانین هوک‌ها** را رعایت کن (بالاترین سطح، فقط در کامپوننت).
9. **منطق را از ظاهر جدا کن:** fetch در `services` ، منطق تکراری در `hooks` .
10. **اول ساده، بعد بهینه.** پیش از وقت بهینه‌سازی نکن.
11. **مدیریت سه حالت:** loading، error، empty.
12. **اسم‌ها واضح و یکدست باشند.** یک غلط املایی (مثل Populer و Popular) تا ساعت‌ها تو را معطل می‌کند.

اشتباهات رایج و راه‌حلشان

علامت	علت	راه‌حل
`JSX expressions must have one parent element`	چند عنصر هم‌سطح بدون والد	همه را داخل یک `div` یا `<>...</>` بگذار
`Missing catch or finally clause`	`try` بدون `catch`	`catch` یا `finally` اضافه کن
صفحه سفید + `useRoutes()` `... Router`	`BrowserRouter` نیست	`App` را داخل `BrowserRouter` در `main.jsx` بگذار
`Cannot read properties of undefined`	prop یا داده هنوز نرسیده یا اسم اشتباه است	`movie?.title` یا بررسی کن prop پاس داده شده
import شکست می‌خورد (`does not provide an export named`)	اسم تابع در دو فایل فرق دارد	اسم‌ها را یکی کن، از F2 (Rename) استفاده کن
استایل اعمال نمی‌شود	`className` با اسم کلاس CSS فرق دارد، یا فایل CSS import نشده	با Inspect بررسی کن؛ import را چک کن
عکس‌ها بزرگ یا کج‌اند	کانتینر grid استایل ندارد	کلاس grid را درست کن؛ برای عکس `object-fit: cover` و `aspect-ratio` بگذار
عکس نمایش داده نمی‌شود	آدرس ناقص است (مثلاً فقط `poster_path` TMDB)	آدرس کامل: `https://image.tmdb.org/t/p/w500` + مسیر
`Warning: Each child in a list should have a unique key`	`key` فراموش شده	`key={item.id}` اضافه کن
تابع یا داده هرگز اجرا نمی‌شود	تابع async تعریف شده ولی صدا زده نشده	بعد از تعریف، آن را صدا بزن
صفحه با هر تایپ حرف رفرش می‌شود	`preventDefault` نیست	در `onSubmit` بنویس `e.preventDefault()`
حلقه‌ی بی‌نهایت رندر	`setState` مستقیم در بدنه‌ی کامپوننت، یا effect بدون آرایه‌ی وابستگی	set را داخل effect یا رویداد بگذار؛ آرایه‌ی وابستگی بده

علامت	علت	راه‌حل
`localhost:5173` کد قدیمی نشان می‌دهد	سرور قبلی هنوز در پس‌زمینه اجراست	آدرس چاپ‌شده در ترمینال را باز کن یا سرور قدیمی را ببند
بعد از Deploy خطای 404 با رفرش	بازنویسی SPA تنظیم نشده	`vercel.json` یا `_redirects` اضافه کن
بعد از Deploy خطای import فایل	تفاوت حروف بزرگ و کوچک اسم فایل	اسم فایل و import را دقیقاً یکی کن
API کار نمی‌کند	کلید اشتباه یا `.env` بعد از تغییر ریستارت نشده	تب Network را ببین؛ سرور را دوباره اجرا کن

چک‌لیست هر پروژه

قبل از شروع

- [ ] مسئله در یک جمله نوشته شد
- [ ] قابلیت‌های MVP مشخص شد
- [ ] صفحه‌ها و آدرس‌ها مشخص شد
- [ ] طرح کاغذی هر صفحه کشیده شد
- [ ] منبع داده (API یا ثابت) مشخص شد

راه‌اندازی

- [ ] پروژه با Vite ساخته شد و اجرا می‌شود
- [ ] قالب اولیه تمیز شد
- [ ] ساختار پوشه‌ها ساخته شد
- [ ] Git راه‌اندازی و اولین commit انجام شد
- [ ] `.gitignore` شامل `node_modules` و `.env` است

ساخت

- [ ] کامپوننت‌ها با داده‌ی ثابت ساخته و نمایش داده شدند
- [ ] استایل پایه و ریسپانسیو انجام شد
- [ ] Router نصب و مسیرها تعریف شد، شامل ۴۰۴
- [ ] state و رویدادها اضافه شد
- [ ] فایل `services/api.js` ساخته و داده‌ی واقعی گرفته شد
- [ ] حالت‌های loading و error و empty ساخته شد
- [ ] کلید API در `.env` است
- [ ] state سراسری (در صورت نیاز) با Context ساخته شد

کیفیت

- [ ] `npm run lint` بدون خطا
- [ ] `npm run build` بدون خطا
- [ ] کنسول مرورگر بدون خطا و هشدار مهم
- [ ] تست دستی همه‌ی سناریوها انجام شد
- [ ] ریسپانسیو در موبایل بررسی شد
- [ ] تگ‌های معنایی و `alt` عکس‌ها رعایت شد

- [ ] اسم فایل‌ها و importها از نظر حروف بزرگ و کوچک یکی است

انتشار

- [ ] پروژه روی GitHub است
- [ ] متغیرهای محیطی در سرویس Deploy تنظیم شد
- [ ] بازنویسی مسیرها برای SPA تنظیم شد
- [ ] آدرس نهایی در موبایل و مرورگر دیگر تست شد
- [ ] فایل README با توضیح پروژه نوشته شد

دستورات و میانبرها

دستورات پروژه

دستور	کار
`npm create vite@latest name -- --template react`	ساخت پروژه‌ی جدید
`npm install`	نصب همه‌ی پکیج‌های package.json
`npm install pkg`	نصب یک پکیج جدید
`npm install -D pkg`	نصب پکیج فقط برای توسعه
`npm run dev`	اجرای سرور توسعه
`npm run build`	ساخت نسخه‌ی نهایی
`npm run preview`	تست نسخه‌ی build شده
`npm run lint`	بررسی کیفیت کد

میانبرهای VS Code

میانبر	کار
`Ctrl + Shift + F`	جستجوی متن در کل پروژه
`Ctrl + P`	باز کردن فایل با اسم
`Ctrl + Shift + O`	رفتن به تابع یا کامپوننت در فایل
`F12` یا `Ctrl + Click`	رفتن به تعریف
`Shift + F12`	دیدن همه‌ی استفاده‌ها
`F2`	تغییر اسم در کل پروژه
`Ctrl + /`	کامنت کردن خط
`Alt + Up/Down`	جابه‌جایی خط
`Shift + Alt + Down`	کپی کردن خط
`Ctrl + D`	انتخاب مورد بعدی همان کلمه
`Ctrl + J`	باز و بسته کردن پنل پایین (ترمینال)
`rafce` + Tab	ساخت کامپوننت آماده (افزونه‌ی snippets)

میانبرهای مرورگر

میانبر	کار
`F12`	باز کردن DevTools
`Ctrl + Shift + M`	شبیه‌ساز موبایل
`Ctrl + Shift + R`	رفرش با پاک کردن کش

بعد از پروژه‌ی اول چه یاد بگیرم؟

به این ترتیب پیشنهاد می‌شود:

1. **پروژه‌ی دوم و سوم را بدون ویدیو بساز** (مثلاً لیست کارها، اپ آب‌وهوا، فروشگاه ساده). یادگیری واقعی همین‌جاست.
2. **JavaScript مدرن را عمیق‌تر یاد بگیر:** destructuring، spread، async/await، promise، map/filter/reduce.
3. **Tailwind CSS** برای استایل‌دهی سریع‌تر.
4. **TypeScript** برای کد امن‌تر و کمتر خطادار.
5. **TanStack Query (React Query)** برای مدیریت حرفه‌ای دریافت داده، کش و خطا.
6. **Zustand یا Redux Toolkit** برای state سراسری بزرگ.
7. **تست:** Vitest و React Testing Library.
8. **Next.js** برای سئو، رندر سمت سرور و پروژه‌های بزرگ.
9. **بک‌اند:** Node.js و Express و یک دیتابیس، برای ساخت API خودت و لاگین.
10. **داک رسمی React:** `react.dev` بهترین منبع و همیشه به‌روز است.

> آخرین توصیه: هر روز کمی کد بنویس. ثبات از تلاش‌های بزرگ و پراکنده مهم‌تر است. اگر در پروژه‌ات گیر کردی، پیام خطا را کامل بخوان؛ بیشتر وقت‌ها خودش جواب را گفته است.